

Ejercicio 7: Arbitraje de Divisas (Programación Dinámica con Logaritmos)

Enunciado

Se denomina arbitraje al uso de las discrepancias en las tasas de cambio de divisas para obtener un beneficio.

Supondremos que hay n tipos diferentes de moneda, y que se dispone de una tabla $C(1..n, 1..n)$ donde $C(i, j)$ es la tasa de cambio entre la divisa i y la divisa j . Se cumple que $C(i, j) = 1/C(j, i)$ y $C(i, i) = 1$.

Diseñar un algoritmo de programación dinámica que dada la matriz C y el identificador i de una divisa, nos proporcione el valor del mejor arbitraje, donde este consiste en una secuencia $i_1, i_2, \dots, i_k$ de identificadores distintos entre sí, y distintos de i , tal que su valor $C(i, i_1) \cdot C(i_1, i_2) \cdot \dots \cdot C(i_{k-1}, i_k) \cdot C(i_k, i)$ es máximo.

1. Transformación con Logaritmos

El objetivo original es maximizar el producto de las tasas de cambio de un ciclo:

$$\text{Maximizar } C(i, i_1) \cdot C(i_1, i_2) \cdot \dots \cdot C(i_k, i)$$

Trabajar con multiplicaciones en programación dinámica presenta dos problemas: el riesgo de desbordamiento (u *underflow* con números muy pequeños) y que no se acopla directamente con los algoritmos estándar de caminos aditivos.

Para resolverlo, aplicamos **logaritmos** a ambos lados. Dado que el logaritmo es una función estrictamente creciente, maximizar el producto es equivalente a maximizar la suma de sus logaritmos:

$$\log(A \cdot B) = \log(A) + \log(B)$$

Por tanto, el problema se transforma en:

$$\text{Maximizar } \log(C(i, i_1)) + \log(C(i_1, i_2)) + \dots + \log(C(i_k, i))$$

Definimos una nueva matriz de pesos aditivos W de tamaño $n \times n$ donde:

$$W[u][v] = \log(C(u, v))$$

Un arbitraje será rentable si esta suma de logaritmos es estrictamente mayor que 0 (lo que equivale a un producto mayor que 1, ya que $\log(1) = 0$).

2. Estructura de Programación Dinámica

Dado que el enunciado exige que los identificadores intermedios del ciclo sean **distintos entre sí** (un ciclo simple de longitud máxima n), limitaremos la longitud del camino a un máximo de n pasos.

Definición de la Función Óptima

Definimos la matriz $DP[u][j]$ como:

- El **máximo valor** (suma de logaritmos) de un camino que comienza en la divisa de origen i , tiene una longitud exacta de j transiciones y termina en la divisa u .

Relación de Recurrencia

Para llegar a la divisa u en exactamente j pasos, debemos venir de alguna divisa previa v en $j - 1$ pasos, y realizar la transición final $v \rightarrow u$:

$$DP[u][j] = \max_{1 \leq v \leq n, v \neq u} \{DP[v][j-1] + \log(C(v, u))\}$$

Casos Base e Inicialización

- Origen:** $DP[i][0] = 0$ (comenzar en i con longitud 0 no cuesta nada).
- Resto:** $DP[u][0] = -\infty$ para todo $u \neq i$ (es imposible estar en otra divisa en 0 pasos).

3. Algoritmo y Complejidad

Pseudocódigo

```
Entrada: C[1..n][1..n] (matriz de cambios), i (divisa inicial)

// 1. Inicialización de la tabla DP de tamaño n x (n+1)
DP[1..n][0..n] = -infinito
pred[1..n][0..n] = -1

DP[i][0] = 0

// 2. Rellenar la tabla DP paso a paso (longitud del camino j)
Para j desde 1 hasta n:
    Para u desde 1 hasta n:
        Para v desde 1 hasta n:
            Si (u != v y DP[v][j-1] != -infinito):
                valor = DP[v][j-1] + log(C[v][u])
                Si (valor > DP[u][j]):
                    DP[u][j] = valor
                    pred[u][j] = v

// 3. Buscar el mejor ciclo que regresa a la divisa de origen 'i'
mejor_valor = -infinito
mejor_longitud = -1

Para j desde 2 hasta n:
```

```

Si (DP[i][j] > mejor_valor):
    mejor_valor = DP[i][j]
    mejor_longitud = j

Si mejor_valor <= 0:
    Retornar "No existe arbitraje rentable"
Sino:
    // Reconstrucción del camino óptimo
    Camino = [i]
    actual = i
    Para j desde mejor_longitud decreciendo hasta 1:
        anterior = pred[actual][j]
        Añadir anterior al principio de Camino
        actual = anterior

Valor_Real_Arbitraje = exp(mejor_valor)
Retornar Valor_Real_Arbitraje, Camino

```

Análisis de Coste

- **Temporal:** $O(n^3)$. Tenemos tres bucles anidados: la longitud del camino j (hasta n), el nodo destino u (n elementos), y el nodo origen de la transición v (n elementos).
- **Espacial:** $O(n^2)$ para almacenar las matrices DP y $pred$ de tamaño $n \times (n + 1)$.